

Larkos Training Loop and Inference Runner

Witold Warchol

June 2, 2026

1 Overview

The Larkos system consists of two major components:

1. **TrainingLoop**: Full differentiable training with gradient computation, emotional state updates, and cognitive fusion
2. **LarkosRunner**: Inference-only forward pass that loads trained weights and runs without gradient accumulation

Both components share the same core modules: the LarkosModel, _FusionTransformerHead, embedding networks, and text codec. The key difference is that TrainingLoop computes gradients and updates weights, while LarkosRunner only performs forward inference using frozen weights.

2 Core Mathematical Components

2.1 Fourier Encoding

The input $x \in \mathbb{R}^D$ is encoded using positional-style Fourier features:

$$\phi(x) = [\sin(2^0\pi x), \sin(2^1\pi x), \dots, \sin(2^{E-1}\pi x), \cos(2^0\pi x), \cos(2^1\pi x), \dots, \cos(2^{E-1}\pi x)]$$

where E is the number of encodings (FOURIER_ENCODINGS). The output has dimension $2 \cdot D \cdot E$.

Rationale: Fourier encodings help the model learn periodic and frequency-dependent patterns. They provide stable gradients compared to raw inputs and help with generalization.

2.2 Online Normalization

Two normalization strategies track running statistics:

2.2.1 OnlineMinMax

Per-dimension running min/max via exponential moving average (EMA):

$$m_t^{(i)} = m_{t-1}^{(i)} + \alpha(x_t^{(i)} - m_{t-1}^{(i)})$$

where $\alpha = 0.02$ (momentum). The normalized output is:

$$\tilde{x}^{(i)} = \frac{x^{(i)} - m}{M - m + \epsilon} \cdot 2 - 1$$

clamped to $[-3, 3]$.

Rationale: EMA-based min/max prevents early outliers from permanently warping the normalization range. The system gradually forgets extreme values and re-centers around the actual distribution.

2.2.2 OnlineMeanStd

Per-dimension running mean and variance via EMA:

$$\mu_t^{(i)} = \mu_{t-1}^{(i)} + \alpha(x_t^{(i)} - \mu_{t-1}^{(i)})$$

$$\sigma_t^{(i)} = (1 - \alpha) \left(\sigma_{t-1}^{(i)} + \alpha(x_t^{(i)} - \mu_{t-1}^{(i)})^2 \right)$$

Output normalization:

$$\hat{x}^{(i)} = \frac{x^{(i)} - \mu}{\sqrt{\sigma} + \epsilon_f}$$

where $\epsilon_f = 0.1$ (std floor).

Rationale: The std floor prevents near-constant dimensions from dividing by zero and blowing up. It keeps low-variance dimensions genuinely quiet rather than amplified by numerical instability.

2.3 Temporal Context Window

The Fourier-encoded input is padded into a temporal window of size `TEMPORAL_WINDOW`:

$$x_{\text{temporal}} = \text{concat}(x_{t-w+1}, x_{t-w+2}, \dots, x_t)$$

where $w = \text{TEMPORAL_WINDOW}$. Earlier positions are zero-padded.

Rationale: Temporal history provides context for the model to detect trends and patterns over time, improving predictions.

3 Training Loop Architecture

3.1 Forward Pass

The forward pass (`_forward`) builds a fully differentiable computation graph:

$$\text{model_pred} = \text{LarkosModel}(x_{\text{temporal}}, \text{embed_ctx})$$

The `LarkosModel` outputs a vector that is split into an LLM embedding (`llm_embed`) and auxiliary dimensions.

3.1.1 Cross-Attention Path

The LLM embedding is gated and fused with text context:

$$\text{embed_gate} = \text{EmbedWeightNet}(x_{\text{norm}})$$

$$\text{llm_embed_g} = \text{llm_embed} \odot \text{embed_gate}$$

$$\text{llm_embed_ca} = \text{InputCrossAttention}(x_{\text{norm}}, \text{llm_embed_g})$$

where $\odot$ denotes element-wise multiplication.

Why Cross-Attention: Instead of concatenating the input and embedding, cross-attention treats the D -dimensional input as D separate 1-D query tokens and the E -dimensional embedding as key/value. This is far more expressive than simple concatenation and allows learned interaction.

3.1.2 Cognitive Fusion

The cross-attention embedding is detached (breaking the gradient graph at the C boundary) and passed to cognitive fusion:

$$\text{fused_cog_raw} = \text{cognitive_fuse}(\text{llm_embed_ca}, \text{neurons}, \text{mem_state}, \text{default_weights}, \dots)$$

Graph Breaking: The detach at the C boundary is intentional. The C-side fusion depends on backend neurons and memory state which are non-differentiable objects. The Python side cannot backprop into them. An auxiliary loss on the cross-attention embedding (`aux_proj`) provides the gradient signal for the modules before the C boundary.

3.1.3 Frozen Input Windows

Training targets are refreshed every `TARGET_FREEZE_INTERVAL` epochs. Between refreshes, both the cached target and cached fused cognitive vector are held constant:

$$\text{frozen_input} = (\text{cache exists}) \wedge (\text{epoch} - \text{target_epoch}) < \text{INTERVAL}$$

Rationale: If the target is fixed but the transformer input drifts, the model chases a moving input toward a fixed point, producing unstable loss. Pinning both together and refreshing them synchronously prevents this pathology. The frozen window allows the model to converge on the pair without the input changing underneath it.

3.2 Fusion Transformer Head

The `_FusionTransformerHead` treats the fused cognitive vector as three contiguous bands that form a 3-token sequence:

$$\text{Band 0} = \text{fused}[0 : \text{BAND_Q}] \quad (\text{LLM query stream})$$

$$\text{Band 1} = \text{fused}[\text{BAND_Q} : \text{BAND_Q} + \text{BAND_N}] \quad (\text{neuron stream})$$

$$\text{Band 2} = \text{fused}[\text{BAND_Q} + \text{BAND_N} :] \quad (\text{memory stream})$$

Each band is projected into a shared d_{model} space:

$$\text{tok}_i = \text{proj}_i(\text{Band}_i) + \text{stream_embed}_i$$

A transformer encoder runs self-attention over the 3 tokens:

$$\text{enc} = \text{TransformerEncoder}([\text{tok}_0, \text{tok}_1, \text{tok}_2])$$

Output is mean-pooled and projected to neuron dimensions:

$$\text{fused} = \text{LinearOut} \left(\frac{1}{3} \sum_i \text{enc}_i \right)$$

Why Three Bands: The bands carry different semantic information (LLM query, neuron state, memory). Using separate projections and stream embeddings lets each stream learn its own mapping and lets attention determine their balance.

Mean Pooling: Pooling instead of taking a single token keeps all three streams in the output. Attention learns to weight each stream appropriately rather than collapsing to one.

3.3 Loss Computation

The loss is a weighted blend of four components:

$$L_{\text{total}} = \lambda_1 L_{\text{outer}} + \lambda_2 L_{\text{base}} + \lambda_3 L_{\text{pred}} + \lambda_4 L_{\text{aux}}$$

where:

$$L_{\text{outer}} = \text{SmoothL1}(\text{maml_pred}, \text{target})$$

$$L_{\text{base}} = \text{SmoothL1}(\text{fused}, \text{target})$$

$$L_{\text{pred}} = \text{SmoothL1}(\text{model_pred}, \text{target})$$

$$L_{\text{aux}} = \text{SmoothL1}(\text{aux_proj}(\text{llm_embed_ca}), \text{target})$$

The weights are:

$$\lambda_1 = \text{mc_blend}$$

$$\lambda_2 = (1 - \text{mc_blend}) \cdot 0.4$$

$$\lambda_3 = (1 - \text{mc_blend}) \cdot 0.3$$

$$\lambda_4 = 0.2$$

where $\text{mc_blend} \in [0.1, 0.5]$ is derived from Monte Carlo variance:

$$\text{mc_blend} = \max(0.5 - \sigma_{\text{MC}}^2 \cdot 0.5, 0.1)$$

Adaptive Blending: High MC variance (model uncertainty) increases the weight on the outer (MAML) loss, which is more stable. Low variance decreases it. The system adapts loss weights based on model confidence.

SmoothL1 Loss: Uses $\beta = 0.1$, giving L_2 near zero and L_1 far from zero. This prevents saturation unlike Cosine+L1 which fought itself.

Auxiliary Loss: The aux path carries gradient signal to cross-attention, embed-weight-net, and text-proj, which sit before the non-differentiable C boundary.

3.4 Gradient Balancing

The `embed_weight_net` branch is often weak. Balancing rescales weak gradients:

$$\text{scale} = \min\left(\frac{\text{norm}_{\text{FT}}}{10 \cdot \text{norm}_{\text{embed}}}, 5\right)$$

where norms are EMA-smoothed to avoid per-epoch pulses.

$$p.\text{grad} \leftarrow p.\text{grad} \cdot \text{scale}$$

All gradients are then clipped globally:

$$\|g\|_2 \leq 1.0$$

Rationale: Prevents any single module from exploding and ensures balanced learning across branches.

3.5 Monte Carlo Sampling

During forward, the model runs in training mode and takes T stochastic samples to estimate output variance:

$$\text{samples} = [\text{model}(x + \epsilon_1), \dots, \text{model}(x + \epsilon_T)]$$

where $\epsilon_i \sim \mathcal{N}(0, 0.05^2) \cdot r_{\text{explore}}$ if exploration is above threshold, else zeros (dropout only).

$$\sigma_{\text{MC}}^2 = \text{Var}(\text{samples})$$

Rationale: Uncertainty estimates inform loss weighting. Without dropout, variance collapses and weights become uniform. Gaussian noise at high exploration keeps estimates meaningful.

3.6 Frozen Skip Logic

On a frozen (input, target) window, once the loss falls below `_frozen_skip_floor` (≈ 0.15), the optimizer step is skipped:

$$\text{skip} = (\text{in_frozen_window}) \wedge (L_{\text{loss}} < 0.15)$$

Rationale: Continuing to step a memorized pair drives gradients to $\sim 10^{-4}$ and wastes progress. Skipping prevents the sawtooth artifact in loss logs.

4 Emotional and Identity Updates

After each backward pass, side-effects update the C-side backend:

4.1 Novelty Derivation

From loss history:

$$\text{novelty} = \min\left(\frac{|L_{\text{current}} - \bar{L}|}{\bar{L} + 10^{-8}}, 1.0\right)$$

where $\bar{L}$ is the mean of the loss history. This is blended with the backend’s reflection:

$$\text{novelty} = \max(\text{novelty}, \text{reflect_novelty}, \text{reflect_drift})$$

4.2 Satisfaction and Emotion

Satisfaction combines low loss and improving loss:

$$\text{sat} = \min(\max(1.0 - L, 0) + \max(L_{\text{prev}} - L, 0), 1.0)$$

Emotion type and strength are picked based on loss delta and novelty:

$$\text{type} = \begin{cases} \text{SURPRISE} & \text{if novelty} > 0.7 \\ \text{LOVE} & \text{if } L_{\text{prev}} - L > 0.03 \\ \text{HATE} & \text{if } L - L_{\text{prev}} > 0.03 \\ \text{SURPRISE} & \text{otherwise (weak)} \end{cases}$$

$$\text{strength} = \begin{cases} \min(\text{novelty} \cdot 0.6, 0.6) & \text{type} = \text{SURPRISE} \\ \min(\Delta L \cdot 3.0, 0.6) & \text{type} = \text{LOVE/HATE} \\ 0.1 & \text{deadband} \end{cases}$$

Deadband: The 0.03 threshold prevents small wiggles from triggering extreme emotions. The 0.1 deadband emotion keeps arousal decaying instead of being pinned high.

5 Inference: LarkosRunner

5.1 Initialization and Loading

LarkosRunner reconstructs only forward-path modules and loads trained weights via `load_checkpoint`:

```
Runner = LarkosRunner(backend, ckpt_path)
```

The C-side backend is reloaded with saved memory and network states:

```
backend.load_memory()
```

```
backend.load_network_states()
```

Rationale: Without the trained backend state, `cognitive_fuse` would see empty neurons and memory, and the readout would not reflect training dynamics. Both halves (PyTorch weights + C-side state) are needed together; neither is usable alone.

5.2 Forward Inference

The runner’s `step` method mirrors `TrainingLoop._forward` but without:

- MC probes (no uncertainty sampling; frozen dropout)
- MAML inner loop (single forward pass only)
- Auxiliary loss computation (output path only)

All computation runs with `@torch.no_grad()`, preventing gradient accumulation and saving memory.

5.3 Input and Frozen Window Handling

Since inference has no training schedule, the frozen-window logic is disabled:

```
frozen_input = False
```

The fused cognitive vector is always fresh (from the live backend) and re-normalized:

```
fused_cog = normalize(cognitive_fuse(...))
```

This mirrors the behavior of a target-refresh epoch in training.

5.4 Output

The runner returns a dictionary with:

- `fused`: The final neuron prediction (NumPy array)
- `model_pred`: Raw LarkosModel output
- `text_input`: The driver sentence
- `text_output`: Decoded text from the fused vector

6 Key Design Decisions

6.1 Why Detach at the C Boundary

The C-side cognitive fusion reads non-differentiable backend objects (neurons, memory, network history). Python cannot build a computation graph through C code. Detaching prevents error and forces the gradient signal through the auxiliary loss instead.

6.2 Why Frozen Input Windows

Updating the target every epoch while the transformer input drifts (due to neuron and memory changes) causes the loss to thrash. Pinning both on the same refresh schedule prevents chasing a moving target.

6.3 Why Separate Projections for Bands

Each band carries different information (LLM embedding, neuron state, memory). Shared projections would force all streams through the same learned mapping. Separate projections (and stream embeddings) allow each stream to learn its own representation and let attention decide balance.

6.4 Why SmoothL1 Loss

SmoothL1 with $\beta = 0.1$ avoids saturation of cosine losses and the fighting behavior of Cosine+L1. It is stable and well-behaved for vector regression.

6.5 Why Auxiliary Loss

The cross-attention, embed-weight-net, and text-proj networks sit before the C boundary. Without a separate loss on their outputs, they would receive zero gradient (the path to target is broken). The auxiliary loss regresses the cross-attention embedding to the target dimension and provides the needed signal.

7 Scheduler and Optimization

Training uses:

- Optimizer: Adam with $\text{lr} = \text{BASE_LR}$, $\text{weight_decay} = 10^{-4}$, $\text{eps} = 10^{-6}$
- Scheduler: CosineAnnealingLR with $T_{\text{max}} = \text{COSINE_T_MAX}$, $\eta_{\text{min}} = 10^{-5}$
- EMA: Exponential moving average of main model weights (used for validation and inference)

The scheduler steps after each backward pass, ensuring a smooth cosine decay over the epoch range.

8 Conclusion

The Larkos training loop and runner form a unified system:

- **Training:** Full differentiable forward pass, multi-component loss, frozen windows, emotional state management
- **Inference:** Fast forward-only pass, loaded weights and backend state, no gradient overhead

Both share core modules and math but are optimized for their respective tasks. The design prioritizes stability (frozen windows, gradient balancing, adaptive loss weights), interpretability (emotional and identity updates), and efficiency (EMA, frozen sampling).